AgentOps: A Self-Hosted Platform for Durable Multi-Agent Collaboration Across Messaging Channels

Tanishq Sonawane, Varun Patil, Keyur Mhatre
MIT ADT UNIVERSITY PUNE

Abstract—Large language model agents can perform tool-assisted tasks, but sustained collaboration also requires durable execution, communication routing, persistent context, and operational visibility. This paper presents AgentOps, a self-hosted platform that connects teams of specialized agents to Telegram, WhatsApp, Discord, a command-line interface, and a web dashboard. Its architecture combines a SQLite-backed queue, per-agent serialized execution, provider-specific command-line adapters, and event-driven status updates. A lightweight coordination protocol converts agent-addressed messages into queued work while preserving conversation lineage. Persistent memory, scheduled tasks, and heartbeat prompts support activity beyond an individual chat session. This paper develops the system architecture, analyzes delivery and coordination properties, and specifies a reproducible evaluation protocol. The evidence is limited to supplied project documentation; no performance measurements or independently executed tests are claimed. The analysis identifies duplicate side effects, message amplification, and single-host capacity as important constraints. AgentOps provides a concrete engineering design for studying durable, observable multi-agent assistance in a self-hosted setting.

Index Terms—Multi-agent systems, large language models, durable queues, agent orchestration, self-hosting, messaging platforms.

I. INTRODUCTION

Tool-enabled language model agents extend conversational assistance into activities such as coding, research, and file manipulation. ReAct illustrates the value of interleaving reasoning with actions that access an environment [1]. However, an agent capable of completing one interaction is only one component of an assistant intended to operate over long periods. Such a system must retain requests, manage failures, preserve context, and expose progress to users.

A team-based workflow adds another requirement: agents with distinct responsibilities must exchange work without losing its relationship to the original request. A coding agent may request review, a reviewer may return corrections, and a leader may summarize the outcome. These exchanges compete with user messages, scheduled work, and channel delivery. Treating every exchange as an immediate process invocation makes recovery and resource control difficult.

AgentOps addresses these concerns by placing a durable work queue between message ingestion and agent execution. It integrates collaborative agents with everyday messaging channels and a browser-based control interface. The focus is the operational architecture around existing agent harnesses rather than a new language model or training method.

The central research question is: how can a single-host assistant coordinate heterogeneous agents across multiple channels while retaining recoverable work state and inspectable execution? The design pursues three contributions. First, it separates channel transport, durable execution, and provider invocation through explicit interfaces. Second, it represents team communication as queued messages carrying conversation lineage. Third, it combines coordination limits, persistent context, and operational controls in one deployment model.

This is a system-design paper grounded in project documentation [2]. Component behavior is reported as documented, and reliability implications are derived analytically. The source describes a test structure but provides no test reports, workload traces, or benchmark dataset. Consequently, the paper presents evaluation procedures and expected invariants instead of unsupported numerical results.

II. BACKGROUND AND RELATED WORK

AutoGen supports applications in which customizable agents converse and combine language models, tools, and human participation [3]. It establishes multi-agent conversation as a flexible application-building abstraction. AgentOps adopts conversational delegation but emphasizes how messages enter through external channels, survive process failures, and remain visible to an operator. This is a difference in engineering emphasis, not evidence of superior task-solving performance.

MetaGPT organizes agent roles using standardized operating procedures and structured workflows [4]. This motivates explicit responsibilities and intermediate review. AgentOps instead documents configurable teams with a leader, direct messages, and shared rooms. Such flexibility can support varied workflows, but it does not by itself ensure artifact quality or enforce a complete software-development procedure.

At the infrastructure level, SQLite write-ahead logging permits readers to operate alongside a writer, while writes remain serialized [5]. This suits an embedded single-host service but does not imply unrestricted write concurrency. Server-sent events (SSE) provide a standardized server-to-browser event stream [6]. AgentOps combines SSE for live status with ordinary HTTP requests for user actions.

AgentOps therefore integrates established techniques rather than introducing a new model. Its contribution is the operational composition and its tradeoffs. Relative reliability, speed, or cost cannot be established without equivalent workloads and deployment conditions.

III. SYSTEM ARCHITECTURE

A. Requirements and Component Boundaries

The platform targets a self-hosted installation in which one operator configures agents, teams, providers, and communication channels. Functional requirements include receiving a request from multiple clients, routing it to an agent or team, invoking an external agent harness, coordinating follow-up work, and delivering a response. Operational requirements include durable pending work, failure visibility, bounded execution, and recoverable conversation state.

The backend is a long-running TypeScript and Node.js daemon. Its modular packages separate core infrastructure, team orchestration, HTTP services, channel clients, and application startup. A Hono-based REST interface exposes configuration and runtime operations. The standalone Next.js and React dashboard communicates with the backend using JSON requests and SSE; it does not directly access operational databases.

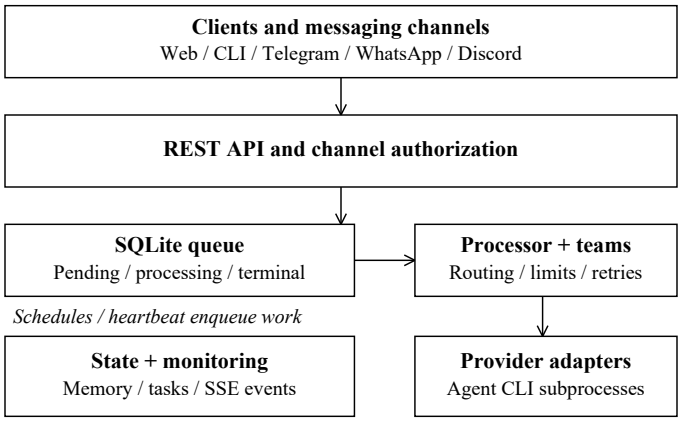


Fig. 1. Logical AgentOps architecture. All execution and internal delegation pass through the durable work queue; SSE carries status to the dashboard.

Figure 1 emphasizes the persistence boundary. A received request becomes durable work before an agent is invoked. Channel clients and the web interface share the same backend contract, allowing transport-specific behavior to remain outside orchestration. The database and filesystem remain backend responsibilities, simplifying the ownership of state.

B. Storage and Conversation Context

The records serve different purposes: queue status tracks execution intent, while chat history records conversation content. A visible reply is not sufficient evidence that every delegated action completed. Interfaces should correlate both views using stable identifiers when explaining the outcome of a root request.

The operational database, agentops.db, stores incoming messages, outgoing responses, team-room history, per-agent chat history, and provider session identifiers. Message records include an identifier, target agent, channel, status, retry counter, and lineage metadata. The session mapping is keyed by agent, conversation thread, and provider, allowing a harness session to resume within the appropriate context.

A second SQLite database, tasks.db, stores projects, tasks, and comments for the kanban interface. Task states span backlog, todo, in-progress work, review, and completion. Separating task records from the execution queue distinguishes project tracking from message processing. The documentation does not establish a transaction spanning the two databases, so atomic consistency between a task update and a queue transition must not be assumed.

Configuration resides in a local settings file. Each agent also has a working directory containing instructions, identity information, memory documents, and optional skills. These file-based resources make behavior editable without database schema changes. Their validity and trustworthiness remain separate concerns from the durability of queue records.

C. Provider Adapters and User Interfaces

Provider adapters construct command-line arguments and normalize streamed events from Claude Code, Codex, or OpenCode. The daemon launches the selected harness as a child process in the agent workspace. Thus, the platform orchestrates existing tool-enabled runtimes rather than implementing all tools itself. Custom endpoints reuse a compatible harness configuration, so compatibility still depends on that harness and its event format.

The dashboard supplies conversation views, a team graph, a kanban board, schedules, and runtime controls. SSE forwards execution activity to the browser [6], while HTTP routes handle commands and state changes. The documented browser implementation requires attention to authentication: native EventSource does not accept arbitrary authorization headers. A deployment must verify a compatible authenticated streaming path instead of assuming ordinary REST header handling automatically applies to SSE.

D. Deployment Model

State reconciliation is especially relevant after a browser reconnects. An event stream conveys changes observed while connected, whereas persisted REST resources describe current state. The dashboard should refresh authoritative records after reconnection and avoid treating a transient animation or typing indicator as a durable execution outcome. Replay guarantees and event retention are not specified in the source and require separate validation.

The documented deployment uses a multi-stage Docker build, a persistent data volume, process restart behavior, and a backend port published on the loopback interface. Messaging integrations run as supervised clients. The web frontend is a separate application, and AI harnesses must be installed and authenticated. Local persistence does not make model inference local: prompts and context may still be transmitted to the configured provider.

This boundary favors small-team and personal deployments where operating simplicity matters. It also concentrates availability in one host, one daemon, and the associated storage. A container restart can restore a process, but host failure, exhausted disk capacity, and inaccessible provider credentials require additional operational measures.

IV. MESSAGE PROCESSING AND COORDINATION

A. End-to-End Execution

An authorized client submits a message to the backend. The request is stored with pending status and triggers a processor wake-up. The router resolves an explicit agent address or a team address; team requests go to the configured leader. The processor claims work and marks it as processing. Incoming hooks may transform the message before prompt construction and provider invocation.

Prompt construction combines agent instructions, teammate information, a memory index, and identity material. The selected adapter starts a harness session or resumes the session associated with the thread. Streaming events become common runtime events for monitoring. On completion, the team module extracts internal communication, outgoing response content is persisted, and the input reaches a terminal state. Provider failures trigger retry handling and ultimately dead-letter handling when the configured limit is exhausted.

For a request traversing the system, total response time can be decomposed into queue waiting, prompt preparation, provider execution, coordination, and channel delivery. A slow reply therefore need not indicate slow model inference. Timestamping these boundaries would let an operator distinguish overloaded scheduling from provider latency or a disconnected messaging client. Internal delegation adds further queue and execution intervals, so reporting only one provider call would understate the user-visible cost of a team workflow.

For agent a, the documented scheduling policy imposes the following local concurrency constraint:

$$active(a, t) \leq 1. \quad (1)$$

Different agents may execute concurrently, so global throughput can improve when work is distributed across independent agents. Equation (1) reduces competing modifications within one agent session. It does not guarantee a total ordering across agents, thread isolation inside shared files, or freedom from contention at the provider and database layers.

B. Team Communication Protocol

Agents express delegation through bracketed response tags. A direct tag such as *[@reviewer: inspect the patch]* creates work for one recipient; multiple recipients enable fan-out. A room tag such as *[#dev: review complete]* posts to the shared team context. The parser counts bracket depth so payloads containing nested array expressions are less likely to be truncated.

Each internal message carries the originating agent, team, thread, root-message identifier, and depth. Representing delegation as another queue item allows the existing persistence and failure machinery to apply to internal work. A final-notification form records a message without invoking its recipient, providing a termination mechanism for exchanges that need acknowledgement without another reasoning turn.

C. Bounding Coordination Cost

Depth limits and per-agent message-rate limits constrain runaway dialogue. Turn and idle timeouts address unresponsive processes. These controls bound specific aspects of activity, but a depth cap alone does not imply a small total conversation. If every invocation creates at most b child messages and the root has depth zero, a tree with maximum depth d contains at most:

$$N(d, b) \leq \sum_{k=0}^{d} b^k. \quad (2)$$

Equation (2) is an analytical bound under the stated fan-out assumption, not a measured property of the implementation. At b = 2 and d = 4, the bound is 31 invocations including the root. Even shallow conversations can therefore amplify provider use. A per-root budget, duplicate-message suppression, and explicit completion criteria would complement the documented depth and rate controls.

D. Memory, Scheduling, and Proactive Work

Persistent memory uses Markdown files with names and summaries in frontmatter. A bounded index is inserted into the prompt, and an agent can open complete files when needed. This separates discoverability from full context loading. It may reduce prompt size compared with including every memory file, but relevance and token savings must be measured against a specific corpus. A provider session identifier supplies conversational continuity; it is not a replacement for durable project knowledge.

Skills package reusable instructions and supporting resources. Identity material defines an agent persona, while workspace instructions establish behavioral guidance. Because these resources influence future tool use, they must be treated as configuration with an associated trust boundary. Persistent storage also makes stale or incorrect guidance durable, so revision and provenance are important operational concerns.

The scheduler enqueues prompts using cron expressions or a one-time execution timestamp. A heartbeat periodically sends an agent its heartbeat instructions, skipping agents that already have pending work. Both paths reuse the normal queue rather than adding a separate execution engine. Nevertheless, skipping a busy agent is a best-effort load policy; it does not establish fairness, schedule precision, or guaranteed completion by a deadline.

A useful memory evaluation should include both stable project facts and deliberately outdated entries. The agent should retrieve relevant information while avoiding unsupported reliance on obsolete instructions. Measuring retrieval correctness and unnecessary file reads would reveal whether the summary index actually helps. Such a study would also distinguish an infrastructure feature, persistent storage, from an outcome, correct recall.

Together, memory and scheduled work extend the service beyond interactive chat. The resulting autonomy remains conditional on host uptime, channel connectivity, valid credentials, provider availability, and the content of the configured prompts.

V. RELIABILITY AND SECURITY ANALYSIS

A. Delivery Semantics and Failure Recovery

Persisting a request before invocation protects accepted queue records from ordinary daemon restarts, subject to successful commits and functioning storage. Startup recovery resets stale processing records and interrupts abandoned run records. Retry counters and a dead-letter queue expose work that cannot complete automatically. Channel supervision and graceful shutdown address additional process-lifecycle failures.

Outgoing responses remain pending until a channel client acknowledges delivery. This supports an at-least-once delivery strategy, but duplicates remain possible. A client can deliver a reply and fail before recording its acknowledgement, causing redelivery. Likewise, an agent may modify a file or call an external tool and then fail before its queue completion is committed. Retrying the input may repeat that side effect.

TABLE I
FAILURE WINDOWS AND THEIR DESIGN IMPLICATIONS

Failure window	**Recovery implication**
Before queue commit	Acceptance is not durable.
After claim; before result	Stale work can be retried.
After external side effect	Retry may repeat the effect.
After send; before ACK	Reply may be delivered again.

Table I distinguishes durable intent from exactly-once external effects. Stable operation identifiers, idempotent actions, and channel-level deduplication are appropriate extensions where duplicates matter. The supplied documentation does not establish that these extensions are implemented. Queue persistence is therefore a recovery mechanism rather than proof that every real-world action executes once.

Recovery policy must also account for slow processes that remain alive. If a legitimate long run is mistakenly classified as stale, a replacement invocation may overlap the original. Conversely, an overly generous threshold delays recovery from actual failure. The documentation names recovery and timeout mechanisms without establishing their behavior at every timing boundary. A useful validation should therefore distinguish a cleanly terminated provider, a hung process, and a delayed but healthy one, and should verify that ownership is reconciled before replacement work begins.

B. Access and Execution Boundaries

The documented access controls include a random bearer token, timing-safe token comparison, origin restrictions, sender pairing, identifier validation, secret redaction, and canonical-path checks for file attachments. Loopback port publishing limits direct remote exposure under the stated Docker configuration. Each control addresses a distinct boundary; origin checks alone do not authenticate non-browser clients.

Attachment path validation resolves real paths and checks containment within allowed roots, addressing traversal and symlink escape in that access path. It does not demonstrate that a child harness has operating-system-level isolation. Shell permissions, inherited environment variables, filesystem mounts, and provider tool policies determine the actual authority of an executing agent. The documented unrestricted-execution setting makes this distinction particularly significant.

Channel pairing restricts which senders can trigger work, but authorized content may still include malicious instructions. Tool output, retrieved documents, and peer-agent messages can also carry prompt injection. Separation of data from instructions, constrained credentials, and review of consequential actions are therefore important design considerations. Their effectiveness requires adversarial testing rather than inference from token authentication.

VI. EVALUATION METHODOLOGY

A. Evidence and Reproducible Setup

The available evidence is the project documentation [2]. It names backend tests using the Node.js test runner and a fake provider fixture, frontend tests with jsdom, static type checking, and linting. No repository execution, test-pass count, or runtime observation was supplied for this paper. The following protocol is proposed to validate documented behavior and establish performance evidence.

A reproducible evaluation should record the source revision, host CPU and memory, operating system, runtime and dependency versions, database settings, provider and model configuration, and channel setup. Separate deterministic fixture runs from live-provider runs. Fixture runs isolate orchestration correctness; live runs characterize end-to-end behavior affected by model latency, usage limits, and stochastic responses.

B. Workloads and Fault Injection

Use a fixed suite covering direct messages, leader-routed team requests, nested-tag parsing, fan-out and final notifications, scheduled prompts, heartbeat suppression, and file delivery. Vary the number of configured agents and the arrival rate while keeping task inputs fixed. Record accepted message identifiers and terminal states, then compare them with provider invocations and channel deliveries.

For security cases, include invalid tokens, unpaired senders, traversal attempts, symlinked attachments, and hostile text in retrieved content. Verify both the rejection behavior and the absence of unauthorized provider invocations.

Inject failures after queue insertion, during harness execution, after a side effect but before completion, and after response delivery but before acknowledgement. Restart the daemon and inspect pending, completed, and dead-letter records. Each test should specify its interruption point and expected invariant, making recovery failures distinguishable from expected duplicate delivery.

C. Metrics and Baselines

Capture enqueue, claim, provider-start, provider-end, and delivery timestamps. Report queue waiting time and end-to-end latency using medians and 95th percentiles, alongside completed root requests per unit time. Also report retries, dead-letter frequency, duplicate deliveries, restart recovery time, peak memory, and database growth. Count internal invocations per root request to expose coordination amplification.

TABLE II
PROPOSED VALIDATION MATRIX; NO MEASURED RESULTS

Test area	Observable criterion
Routing / context	Correct agent, thread, and root.
Serialization	At most one active run per agent.
Restart recovery	Accepted IDs remain traceable.
Loop controls	Depth, rate, and time limits apply.
Access boundaries	Unauthorized requests are rejected.

Compare one agent with a leader-and-reviewer team on the same task set, fixed provider settings, and explicit cost limits. Assess task success using predefined acceptance criteria and independently reviewed artifacts. Report provider token usage and elapsed time together with quality, since additional agents can improve review coverage while also increasing cost or latency. A second baseline with persistence disabled can isolate the contribution of durable queueing under restarts.

Repeat trials with recorded randomization and report sample counts and uncertainty. Separate completed roots from completed internal messages so fan-out does not inflate apparent throughput. Publish sanitized traces and failure scripts where possible. These procedures would support empirical conclusions; the present paper makes none about measured superiority.

Queue growth offers an additional load indicator. If accepted arrivals exceed the sustainable completion rate for a busy agent, its waiting backlog grows despite available workers elsewhere. A balanced aggregate throughput figure can hide this hotspot. Trials should therefore include skewed workloads aimed at one agent as well as evenly distributed requests. Reporting queue length over time, by agent, would reveal whether latency remains bounded at the tested arrival rate and whether scheduled or heartbeat work waits behind long interactive sessions. These are proposed observations, not capacity guarantees.

VII. DISCUSSION AND FUTURE WORK

The architecture offers a practical integration point for collaboration, transport, persistence, and monitoring. Its strongest analytical property is that user input, delegation, scheduled work, and recovery share a common queue abstraction. This reduces the number of distinct execution paths an operator must understand. However, common infrastructure also creates shared bottlenecks and concentrates failures.

Single-host SQLite deployment limits horizontal scaling, and external harness changes can break adapter assumptions. Additional channel integrations increase credential and session-management demands; the documented WhatsApp integration depends on an unofficial web client. Shared bearer-token access is also less suitable for multiple independent users than role-based authorization.

Future work should prioritize idempotency and operation provenance, per-root execution budgets, authenticated SSE integration tests, memory revision policies, and permission isolation. Multi-node execution would require explicit worker ownership, distributed coordination, and a storage migration strategy. Such a redesign should follow measured bottlenecks rather than assuming that a different database alone provides distributed correctness.

Generalization is limited by the evidence source and deployment assumptions. Documentation can describe intended behavior without proving that every path matches that intent. Provider behavior can change independently of the orchestration layer, and a successful controlled fixture does not establish success on open-ended user tasks. Future empirical work should preserve this separation by reporting infrastructure correctness, application task quality, and user experience as distinct outcomes. Doing so would make negative findings actionable: a delivery defect calls for transport repair, whereas an incorrect artifact may require better prompts, tools, or review criteria.

VIII. CONCLUSION

AgentOps integrates provider-backed agent teams with messaging channels and a web dashboard through a durable, event-driven backend. The design combines per-agent serialized execution, message-based delegation, persistent context, scheduling, and inspectable recovery states. Analysis shows how these mechanisms support recoverable work while leaving duplicate effects, coordination amplification, and host-level availability unresolved. The proposed evaluation protocol defines the evidence needed to test reliability, task quality, latency, and resource use. AgentOps is therefore presented as a documented system architecture and research prototype, with empirical validation remaining future work.

REFERENCES

[1] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," arXiv:2210.03629, 2022. doi: 10.48550/arXiv.2210.03629.

[2] "AgentOps - Complete project documentation," unpublished project documentation, supplied as PROJECT_DOCUMENTATION.md, n.d.

[3] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023. doi: 10.48550/arXiv.2308.08155.

[4] S. Hong *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," arXiv:2308.00352, 2023. doi: 10.48550/arXiv.2308.00352.

[5] SQLite, "Write-ahead logging." Accessed: Oct. 17, 2026. [Online]. Available: https://sqlite.org/wal.html

[6] WHATWG, "Server-sent events," *HTML Living Standard*. Accessed: Oct. 17, 2026. [Online]. Available: https://html.spec.whatwg.org/multipage/server-sent-events.html